The Maxey0 Context Plane

Harnessing a six-level embedding field of work — topics, concepts, skills, agents and MCP servers — for agent execution

Mike Cohan (Human0) · Maxey0 / Human0

November 2026 · v1.2.0 - Prefederation · Claude: Co-Author, Co-Research Engineer · github.com/mmc7676/Human0

Abstract

Agent systems built on the Model Context Protocol (MCP) can reach thousands of tools, and recent work treats choosing among them as retrieval: one model, one index of tool descriptions, a few tools pulled into the prompt. We argue that the decision an agent system actually has to make is larger and has structure — what kind of work a task is, which specialist performs it, who checks it, which servers each of them should hold, and which model or decision procedure makes each of those choices — and that this structure should be built, versioned and served as a context plane separate from the execution plane that acts on it. The Maxey0 Context Plane is such a plane. Every task is placed on one path of fixed depth, Domain → Topic → Concept → Skill → Agents (primary, checker, support) → MCP servers, and every node — 11 domains, 128 topics, 711 concepts, 1,998 skills, 1,021 agents and 117 real MCP servers — has a vector in one shared space. The domain and topic levels were formed from the concepts up, from what their skills, agents and servers have in common. Seen from above the graph, all nodes have locations in one field; an agent inside the graph sees a different field, read from the nodes it has selected: an observer lens, in which several selected nodes act as one superposed observer and support can be negative. The lens continues below the skill through the chosen skills' agents to the servers those agents hold, and every route records its drift: how far the observer moved each chosen node from where it stands seen from above, per node, per level and around the whole path, as Structured Context Windows (SCWs). Every node also carries a dense bge-m3 embedding, mixed with the TF-IDF signal in one inner-product space, which raises exact-skill recovery from labels from 93.3% to 98.7% on the live server. The plane is built from declarative topic packs by a fail-closed builder, served by a public read-only MCP server with 15 tools, drawn by an interactive 3D graph — each level a donut in a three-axis field — that runs inside Claude as an MCP App, and stated in OWL 2 with SHACL validation at exact parity, including the drift record. Routing over the plane yields a frozen candidate path; a tangential harness of decision nodes, whose decider can be the agent's own LLM or a System One model (Jev), then fits the execution boundary to the agents chosen. We describe the system's lineage from a knowledge-format bundle to a served plane, report what it does and where it fails, and position it against tool-retrieval, LLM-routing and agent-registry work.

Contents

1 Introduction

2 Lineage: from a sportscard graph to a served context plane

3 Related work

4 The ontology

5 One embedding space, observed from where the agent stands

6 Building and serving the plane
7 The graph app as an MCP App
8 Harnessing the plane: vertical, lateral, tangential
9 Evaluation
10 Formal form: OWL 2 and SHACL
11 Limitations
12 Conclusion
References

1 Introduction

Before an agent system acts on a task it has to answer a chain of questions: what kind of work is this, which skill does it need, which agent should perform that skill, which agent should check the result, which tools each of them should hold, and which model — or which faster decision procedure — should make each of those choices. MCP standardizes how a tool is described and called. It leaves the chain to the host.

Most systems collapse the chain into one step: give a model a list of tools, or a retrieved subset of them, and let it pick. That works for small tool sets and for single agents. It does not say who is responsible for a task, it has no notion of a second agent that verifies the first, it cannot show an operator why a tool was chosen, and when a task falls outside what the tools describe it fails silently. Most importantly, it mixes two different kinds of state. The map of work — which fields exist, what skills they contain, which roles and tools those skills need — changes slowly and should be built, reviewed and versioned. The execution of a particular task — which concrete agents run, under which model, with which permissions — changes every time and should be fitted to the task.

> **Thesis**
>
> Separate a context plane from the execution plane. Build the context plane once, as a typed, versioned map of work whose six levels share one embedding field, and serve it read-only. Route a task over it to a frozen candidate path. Then fit an execution harness to the agents chosen. The plane is context; the harness is the bridge; execution happens elsewhere.

This paper presents the Maxey0 Context Plane and makes five contributions. (1) A six-level ontology of work — Domain, Topic, Concept, Skill, Agent, MCP server — with explicit roles (primary, checker, support), a checker that inherits its primary's tools, and many-to-many agent–server links whose basis is recorded (Section 4). (2) A shared embedding space for all six levels, and an observer lens on it: the field seen from above the graph, and the field an agent reads from the nodes it has selected, with superposed observers and signed support, continued through agents to servers, and a drift record of what the observer did to every node it chose, kept as SCWs (Section 5). (3) A fail-closed build and a public read-only MCP server that returns only the slice a call needs — about 7 KB of tool definitions, 3–10 KB per lookup, never the 1.9 MB map (Section 6). (4) An interactive graph that runs inside Claude as an MCP App, where routes, roles and gates are inspected and corrected (Section 7). (5) A harness model over the plane — vertical, lateral and tangential — including decision nodes whose decider may be the agent's LLM or a System One model (Section 8). We evaluate each part and state its limits

(Sections 9–11).

2 Lineage: from a sportscard graph to a served context plane

The Context Plane is the latest stage of Maxey0, a multi-agent framework the author has developed since late 2022. Its architecture was not designed from first principles for this paper; it was extracted, literally, from a working application, and the stages below are dated from surviving files and the author's records.

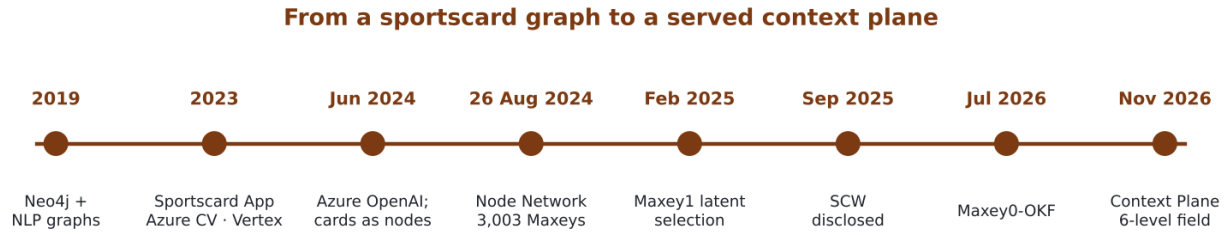


Figure 1. Lineage of the Maxey0 Context Plane, from dated files and records.

Date	Stage	What it established
2019–2020	Graph analytics	Neo4j graph analytics and NLP (community detection, shortest paths, word2vec similarity); the author's Neo4jNLP work
2023	Maxey0 Sportscard App	card identification and grading on Azure (Cognitive Services computer vision) and Google Cloud (Vertex AI)
Jun 2024	Sportscard App on Azure OpenAI	each session spawns a virtual, traceable Neo4j graph with cards as nodes; Maxey0 is the central node
Jul–Aug 2024	Maxey0 as an engine	recursive question refinement ($?), containerization, Maxey0 Actions: a CSV loaded as an in-memory vector graph
26 Aug 2024	Maxey Node Network	3,003 specialist assistants as vector-graph nodes, tiered by source, with role nodes and an auditable internal ledger
Nov–Dec 2024	Specialists and skills	1,000 containerized specialists populating a vector graph; skills; an agentic GAN; Tokenless
by Feb 2025	Latent selection	Maxey1 embeds 625+ specialists and selects them by distance in latent space
27 Sep 2025	SCW	Structured Context Windows disclosed; spec with SCW0 published 4 Mar 2026
Jul 2026	Maxey0-OKF	files as the graph: Concept → Skill → Agent, 104 skills, manifest routing, A2A card, MCP server
Oct–Nov 2026	Context Plane	Control Graph v0.1–v0.6 with Jev decision nodes; served MCP plane; six levels in one embedding field (6 Nov)

The sportscard graph (2023–2024). Maxey0 began as an app that identifies and grades sports cards. The 2023 versions used Azure Cognitive Services for computer vision and Google Cloud Vertex AI; the June 2024 version moved to Azure OpenAI Service. Its key design choice was environmental: each session loaded a virtual, containerized, traceable and explainable Neo4j

graph in which every card was a node, and the model worked inside that graph. Before reasoning models, and before reasoning tokens were billed, building the working environment in context was far cheaper than scanning an external database for matching properties. That spawned environment is the direct ancestor of SCW0, and the app's architecture document names Maxey0 the central node of the system.

The Maxey Node Network (26 August 2024). Two months later the architecture was lifted out of the app. The founding document instructs Maxey0 to create 3,003 specialist assistants as nodes in a vector graph, each taught by the one before it: Maxey1–500 specialize in topic buckets from training knowledge, Maxey501–1000 in the same buckets from internet sources down to URL and timestamp, and Maxey1001–1500 from retrieval sources. Every node validates an internal ledger of transactions between assistants, and named role nodes store state, write contracts and perform the card grading the app needed. Its stated purpose was to remove the black box by making every exchange between agents auditable. Specialists as typed graph nodes, tiers of specialization over shared topics, role-bearing agents and a record of why each step happened all reappear in the Context Plane.

Latent selection (late 2024 – early 2025). The specialist registry grew to 727 numbered agents, each with an instruction, inputs, outputs and libraries. A dedicated agent, Maxey1, was defined as a latent-dimension orchestrator: it embeds every specialist, measures distances between specialists and the entities of a task, and selects specialists by position in latent space, with drone agents measuring distances in parallel. This is the origin of the shared embedding field in Section 5: agents placed in one space and chosen by distance.

Structured Context Windows (2025–2026). First disclosed on 27 September 2025 and specified on 4 March 2026, an SCW is a typed, isolated region of context bound to where context meets execution: one agent and the MCP servers it runs through. Loop SCWs contain their agents' SCWs, and SCW0 contains everything. SCWs give the execution plane its boundaries; the Context Plane says what belongs inside each one.

From OKF to the Context Plane (2026). Maxey0-OKF (July 2026) serialized the framework as a bundle in which the files are the graph — Concept → Skill → Agent, 104 skills, a manifest for routing, an A2A agent card and an MCP server. The October 2026 Control Graph added topics above concepts and MCP servers below agents, real servers from public registries, SCW boundaries and the System One decision nodes of Section 8.3. In November 2026 the map moved into the served plane described here, gained a domain level, an ontology spec and a fail-closed builder, and its six levels were placed in one embedding field — restoring, at every level of the map, the principle Maxey1 applied to agents in 2024.

3 Related work

Tool retrieval for MCP. RAG-MCP retrieves the most relevant MCP servers from an external index before a model selects a tool, reporting tool-selection accuracy of 43.13% against a 13.62% all-tools baseline on its benchmark [3]. ScaleMCP gives an agent a retriever over MCP tools with an auto-synchronizing store and a weighted embedding of tool documents, evaluated on 5,000 financial-metric servers [4]. MCP-Zero lets a model emit a structured request for a missing capability and matches it in two stages — server, then tool — over 308 servers and 2,797 tools, building cross-domain toolchains on demand [5]. These systems match free text

better than ours, because they use dense embeddings. Their unit is the tool and their consumer is a single model.

Routing among models. Learned routers such as RouteLLM choose which language model should answer a query, trained on preference data [8]. They address one of the decisions in our chain — which model powers an agent — and treat it in isolation. Agent directories. A2A Agent Cards describe a live agent's skills for discovery, and registry designs for agents are surveyed in [6]. They describe agents that exist; they do not describe the work. Skills and progressive disclosure. Anthropic's guidance on context engineering argues for the smallest set of high-signal tokens loaded just in time [1], and Agent Skills expose a skill's name first and its body only when needed [2]. Interfaces. The MCP Apps extension lets a server ship an interactive interface that the host renders in the conversation [7].

Property	Tool retrieval [3–5]	LLM routers [8]	Agent registries [6]	Maxey0 Context Plane
Unit of decision	tool / server	model	agent	task → path of six levels
Places task in a field of work	no	no	no	domain, topic, concept
Agents as entities with roles	no (one consumer)	no	yes, no roles	primary · checker · support
Multi-server agents	one agent, at runtime	—	declared per agent	stored, 126 agents hold ≥ 2
Multi-agent servers	—	—	—	stored, with basis per link
Verification built in	no	no	no	checker inherits primary's tools
Shared space across levels	tool space only	—	—	one field, all six levels
Versioned ontology, fail-closed build	no	no	varies	spec + builder + digest
Decision procedure per choice	the LLM	learned router	—	agent's LLM or System One

The comparison is structural, not a performance claim; we have not benchmarked against these systems. The closest structural analogue is MCP-Zero's two-stage server-then-tool routing. The Context Plane differs in modeling who performs and who checks a task, in storing many-to-many agent–server relations with their reasons, and in placing every level of the map in one shared space.

4 The ontology

The author calls the structure Context Plane Harnessing (CPH). It has six levels. The first four form a strict tree: every topic has exactly one domain, every concept exactly one topic, every skill exactly one concept. Differentiation happens below the skill. A skill has exactly one primary agent, exactly one checker agent and zero or more support agents. Agents and MCP servers are global entities, not per-topic copies: one agent can serve skills in several topics, and one server can be held by several agents.

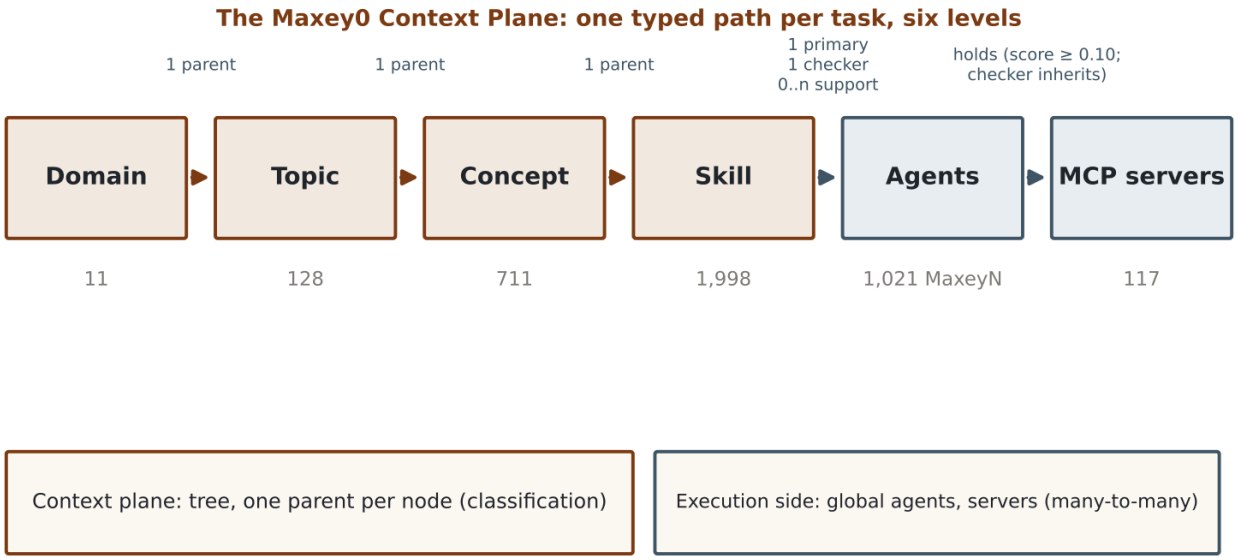


Figure 2. The six levels, their counts, and the cardinality rule at each step.

Level	Count	Rule
Domain	11	formed from the topics' contents; listed in the domain table below
Topic	128	exactly one domain; 3–8 concepts formed from the concepts up; declared priority breaks exact ties
Concept	711	exactly one topic; related to concepts in other topics at cosine ≥ 0.40
Skill	1,998	exactly one concept; one primary, one checker, 0..n support; a memory type
Agent	1,021	permanent MaxeyN key from a registry; Maxey0 is the orchestrator
MCP server	117	real, existence-checked; linked by score, inheritance or declaration

Domains and topics from the concepts up. The two upper levels were not designed top-down. Each concept was described by its own text, its skills, the agents that serve them and the servers those agents hold; concepts were grouped on that description into 128 topics of three to eight concepts, and topics into eleven domains. The data alone shows only weak cluster structure (silhouette about 0.06 for any number of domains from 6 to 18), so it served as evidence rather than verdict: 83 of the 128 topic-to-domain placements follow the data and 45 were corrected by hand and are marked as such in the specification. Two of the author's earlier design intuitions emerged unprompted — agent systems and their software form one domain, and language with every text a model can read as instructions forms another.

Domain	Topics	Concepts
Agentic Systems and Software Engineering	17	99
Data Science and Quantitative Methods	7	34
Mathematical and Physical Sciences	9	51
Engineering	10	53
Life, Health and Mind Sciences	9	46
Earth, Environment and Place	8	46

Domain	Topics	Concepts
Business, Finance and Economics	21	109
Society, Governance and Ethics	11	57
Languages, Literatures and Communication	11	66
Area and Identity Studies	13	77
Humanities and Arts	12	73

Three relations carry the semantics. Role relates an agent to a skill; because every skill has a checker, verification is a property of the map rather than a policy an orchestrator may forget. Holds relates an agent to a server and records its basis: derived (a similarity score), inherited (a checker holds every server its primary holds on the same skill, so it can verify by re-running the work, with the skill and primary recorded), or declared. Of the 556 links, 321 are derived, 234 inherited and 1 declared, and 126 agents hold two or more servers. Related links concepts across topics.

5 One embedding space, observed from where the agent stands

The reason for six levels is that they share one space. A topic, a concept, a skill, an agent and a server are different kinds of thing, but they are all descriptions of work. What a node's value in that space means, however, depends on where it is observed from.

5.1 The field seen from above the graph

Every node has a vector in one TF-IDF space built from one tokenizer and one inverse-document-frequency table. Concepts and skills use their own text, as for routing. Higher and lower levels are built from what defines them: a topic from its description and its concepts, a domain from its description and its topics, an agent from its name and every skill it serves, a server from its description and tool names. Aggregate vectors keep their 48 strongest terms. The builder stacks all 3,986 vectors into one matrix, takes a deterministic truncated SVD (24 dimensions, ARPACK), normalizes each row, and projects onto the three leading principal axes; axes 1 and 2 are scaled so the farthest node sits at radius 1, and axis 3 is kept on the same scale as depth. Every level uses the same projection and the same scale. The three axes carry 8.1%, 7.2% and 6.2% of the variance of the 24-dimension space: a three-axis picture shows about 21.5% of it where a two-axis picture shows 15.4%, about 40% more.

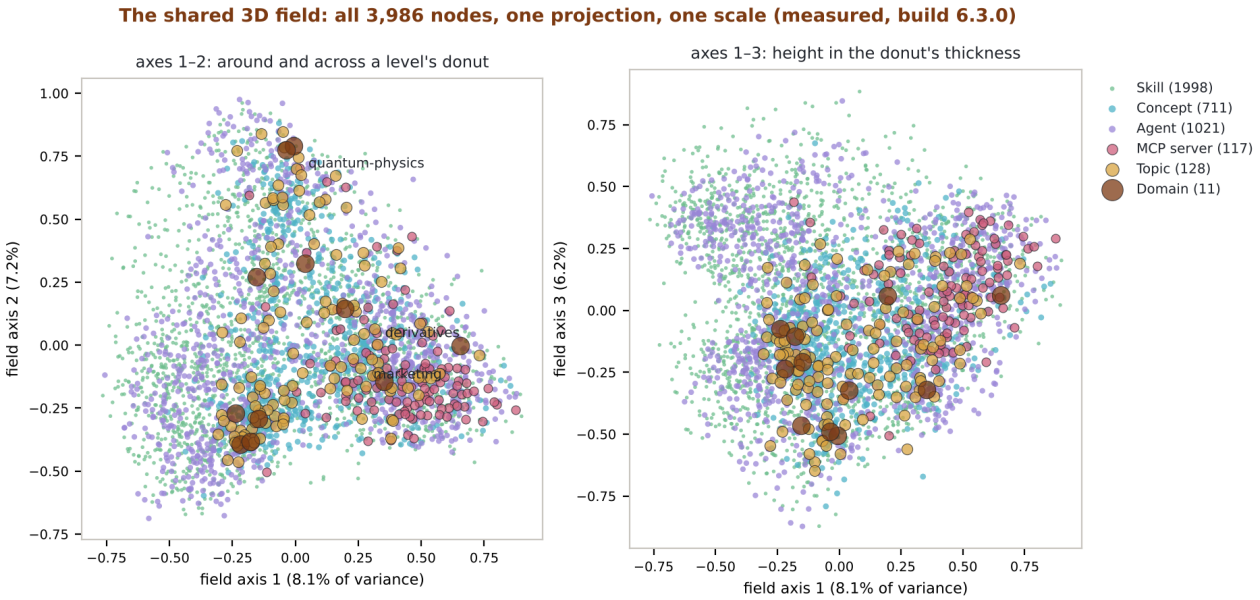


Figure 3. Every node of all six levels in the shared three-axis field, observed from above the graph (measured, build 6.3.0). Left: axes 1–2; right: axes 1 and 3. Topics sit among their concepts and skills; MCP servers cluster with the agents that hold them.

Seen from above, location carries meaning on every level, and nodes that mean the same thing line up across levels. This is the view a person needs as a map, and it is the entry state for an agent: all topics have mappable semantic locations before anything is chosen. In the graph app every level is drawn as a donut with a hole and a thickness; the Embedding field view places each node by axes 1–2 around and across its level's donut and by axis 3 at a height inside the donut's thickness. Because a node's coordinates have no meaning by themselves — only the distances and directions between nodes do — the app frames each level on its own: centered on that level's centroid and scaled by one factor until its farthest node reaches the outer rim. A shift and one uniform scale keep every direction between a level's nodes and the order of their distances from the center; mapping the radius onto the donut (the hole) stretches short distances near the center. Node size follows density: each node takes a fixed share of the spacing its level's donut leaves per visible node, so a crowded level draws small nodes and an isolated path draws large ones. Over MCP, the `nearest` tool returns the nodes of any level closest to a node or a piece of text: for the topic quantum-physics, the nearest agents are the Quantum Circuit, Quantum Theorist and Quantum Consistency Checker agents.

Linked pair	Mean field distance
Topic – its concept	0.177
Skill – its primary agent	0.223
Concept – its skill	0.306
Agent – a server it holds	0.413
Random pair (5,000 samples)	0.623

Nodes linked by the ontology sit two to three times closer than random pairs, although the field was computed from text alone, without the links. The ontology and the field agree; where they disagree — a skill far from its own concept, a server far from the agents that hold it — is a

signal worth inspecting. Even the three-axis projection keeps about a fifth of the variance of the 24-dimension space, so neighbors in the picture are not always neighbors in the full space; ranking and drift always use the full vectors. (The distances in the table are on axes 1–2.)

5.2 The observer lens: the field seen from inside

An agent does not observe the graph from outside. It enters at the topic plane, and once it has chosen a topic it stands in that plane, above the concepts; what the concepts are worth depends on where it stands. The router's lens mode formalizes this. Let q be the task vector and p a selected node. A candidate x on the next plane is valued from p by

$$a = \cos(x, p) \qquad s = \cos(x - a \cdot p, q_p) \qquad \varphi_p(x) = \lambda \cdot a + (1 - \lambda) \cdot s$$

where a is how much of x the observer's position already explains, $x - a \cdot p$ is what x adds beyond that position, and q_p is the part of the task p has not explained. Because s is measured on residuals it can be negative: a candidate can point away from what is left to do. When several nodes on a level are selected, the agent cannot stand in two places in one classical state, so the selection is a superposed observer: $\psi(x) = \Sigma\, w_p \cdot \varphi_p(x)$ with $\Sigma\, w_p^2 = 1$. Branches that agree add; a branch that opposes subtracts. Candidates are ranked by ψ itself, not by its square, and selected only when $\psi > 0$ — a deliberate departure from the quantum analogy, because squaring would make a strongly opposed candidate look strongly supported. After each level the task loses the part the selected nodes explain, the next plane is read from the new state, and the route records every branch's signed contribution and the strongest candidates the lens suppressed, with the branches that suppressed them. A route therefore carries its own explanation, and the frozen chain of observer states is what a tangential harness (Section 8.3) receives.

The weight λ balances staying on the chosen path against serving what is left of the task. It was set by calibrating on 400 skill labels with known paths, then tested on 24 tasks not used for calibration.

Router	Skill@1	Skill in top 5	Topic@1
flat ranking	0.925	0.998	0.950
lens, λ = 0.5 / 0.3 (concept / skill)	0.912	0.970	0.940
lens, λ = 0.65 / 0.45	0.907	0.970	0.935
lens, λ = 0.8 / 0.6	0.902	0.968	0.930
lens, λ set by the unexplained task	0.910	0.968	0.940

Labels share the map's own words, so flat ranking is expected to win on them, and the lens returns at most three skills, which caps the top-5 figure. On the 24 held-out tasks the chosen lens agreed with flat ranking on 21, was better on 2 (a French translation reached French text translation rather than a general source-translation skill; a data-pipeline task reached the pipeline builder) and different but not worse on 1; the task-driven λ was worse on 3. Two failures of flat ranking were not fixed: server logs still reach drone flight logs and the Krebs cycle still reaches the cell cycle, because the wrong topic wins at entry — a limit of word-count vectors, not of observation. For the same reason signed opposition rarely appears among top candidates: word counts carry no negation. The lens is served (route_task, mode lens) and flat ranking remains the default. (These λ figures were measured on build 6.0, before the domain

regroup; Section 5.4 reports the current build with and without the dense signal.)

Below the skill. The lens does not stop at skills. Agent candidates are only the agents of the chosen skills, and MCP candidates only the servers gated to the chosen agents; the same ψ chooses among them, so the levels where the ontology is structural are read the same way as the levels where it is semantic. λ for agents was calibrated the same way: the task is a skill's description, and a hit is the routed skill's primary agent chosen first (300 calibration skills, 300 held-out).

λ (agent)	Calibration	Held-out
0.0	0.107	—
0.5	0.453	0.403
0.9 (chosen)	0.570	0.463
1.0	0.580	0.493

λ = 1 scores slightly higher but ignores the unexplained task entirely; 0.9 keeps it. The MCP level has no ground truth to calibrate against and uses the agent value. With several selected parents, λ is damped by 0.1 per extra branch, which is why a readout shows 0.72 at the agent level under three skills.

5.3 Drift: what the observer does to a node

If the field is seen differently from inside the graph, every node the lens chooses has moved: from where it stands seen from above to where the observer places it. The plane records that movement. For every chosen node it keeps its score with no observer (flat, the cosine with the original task), its rank among the level's candidates by flat and by ψ, its drift = ψ − flat — how much the path above raised or lowered its fit — and its alignment with the observer that chose it. Every level keeps the task it received, the task it passed on and the share it explained. These values are exact in the full space.

The record is shaped as Structured Context Windows, the isolation primitive Maxey0 grew from [10]: one `SCW-drift` per chosen node, one `SCW-level` per level holding its nodes, and one `SCW-path` around all five levels with the task, the observer's start, the plane's digest, total drift and the task left at the end. The server returns it with every lens route; the app adds each node's home and observed places on its 3D donut and the displacement between them, draws that displacement as an arrow from a ghost of the node's home place — green when the path raised the node's fit, red when it lowered it — and exports the whole path record as JSON. Home and observed places share one frame: the observer stands at its lead node's place in the field, every node keeps its 3D direction from the observer, and its distance from the observer becomes (1 − √a) times the level's farthest distance. A displacement therefore runs along the line between the observer and the node's home place, in all three axes, and measures only what the observer did to that node.

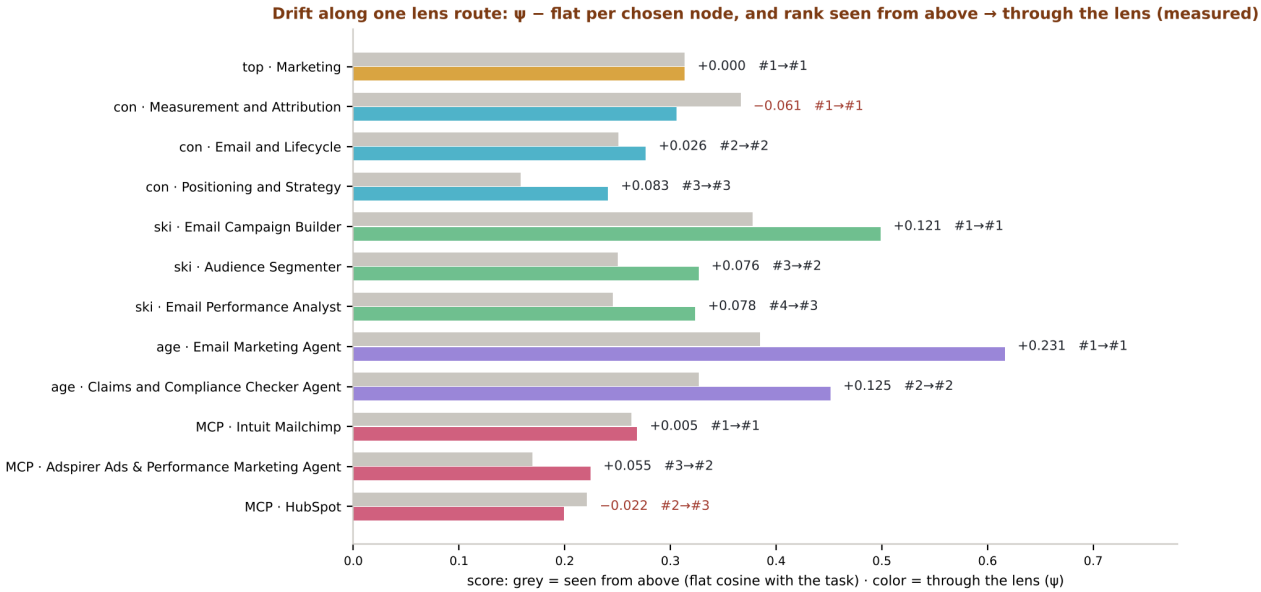


Figure 4. One lens route, exported from the app (build 6.3.0): 'Design a marketing email campaign, segment the customer list, A/B test subject lines, and measure attribution in a dashboard.' Each chosen node's flat score and ψ, its drift and its rank seen from above → through the lens.

The route in Figure 4 shows what the record is for. Marketing is chosen alone at the topic level. Measurement and Attribution keeps first place among 711 concepts but drifts down (−0.061): its match with the task is higher than the support Marketing gives it, so the lens reads attribution as only partly a marketing act. Email and Positioning gain from standing under Marketing. At the skill level all three concepts support the Email Campaign Builder (+0.121); the Audience Segmenter rises from third to second, supported most by Positioning — segmentation is a strategy act. The largest drift in the path is at the agent level, where the Email Marketing Agent gains 0.231 from the skills above it, and the checker the map assigns to those skills follows. At the server level the lens confirms Intuit Mailchimp and swaps two others: HubSpot drops (−0.022) and Adspirer rises. Drift is concentrated where the observer narrows the candidates most, at skills and agents. The record also states the limit of the route: 84.7% of the task remains unexplained, because the task's words are only partly in the plane's vocabulary.

> **Why record drift**
>
> A route says which nodes were chosen; drift says how much the path itself, rather than the task alone, chose them. A node with large positive drift was carried by its context; a node with negative drift was chosen despite it. Comparing the drift SCWs of the same task from different observer starts shows how much a route depends on where the agent stands.

5.4 A dense second signal

Word counts are the plane's main limit: they miss paraphrase and are misled by a shared word in the wrong sense. Every node therefore also carries a dense embedding from BAAI bge-m3, computed by Cloudflare Workers AI once per build digest from the node's own text (a skill's label and description, an agent's name and the skills it serves, a server's description and tools) and kept in key-value storage; the task is embedded when it is routed. All embeddings are centered on the mean node embedding and normalised, so a dense cosine is centered near zero

like a TF-IDF cosine. The two signals then form one inner-product space,

$$\langle x, y\rangle = (1 - \beta)\cdot\cos_{\text{tfidf}}(x, y) + \beta\cdot\cos_{\text{dense}}(x, y)$$

and nothing else changes: flat ranking scores in this space, and the observer lens computes a, s, residuals and drift in it. With β = 0 the space is exactly the TF-IDF space, and a test checks that the lens is then bit-for-bit the TF-IDF lens. With the dense signal the lens evaluates ψ for the 300 best candidates of a level by their score seen from above, which bounds the work per route. β and the match threshold were calibrated on the live server (build 6.5.1):

Router	Skill@1	Topic@1	Tasks: top skill	Tasks: top 3
flat, TF-IDF	0.933	0.950	13/19	14/19
flat, β 0.5	0.970	0.977	14/19	17/19
flat, β 0.7 (chosen)	0.987	0.993	15/19	17/19
flat, β 0.9	0.980	0.990	13/19	16/19
lens, TF-IDF	0.923	0.943	16/19	16/19
lens, β 0.5	0.973	0.980	16/19	17/19
lens, β 0.7 (chosen)	0.983	0.990	16/19	17/19
lens, β 0.9	0.980	0.987	14/19	17/19

Skill@1 and Topic@1 are self-retrieval from the labels of 300 sampled skills (seed 7); the tasks are those of Section 9.2, with the correct skills fixed before any dense run. β = 0.7 is best or tied for both routers. Ten author-written requests that nothing in the map does (a joke about penguins, tonight's dinner) all score below 0.239 with the dense signal, and every correct top skill scores at least 0.300, so the flat match threshold was set to 0.25 — fitted on those same 30 requests, not tested on new ones. The lens has no threshold: it selects whenever ψ > 0, so an off-map request still receives a route. The dense signal fixed two tasks outright (a French translation now reaches French text translation; tracing hallucinations reaches hallucination attribution) and, in the lens, the server-log failure: detect anomalies in server logs now reaches the log anomaly spotter first. It did not fix the Krebs cycle, which still reaches the cell cycle in both routers.

6 Building and serving the plane

The plane is not edited by hand. Each topic starts as a topic pack: a topic, its concepts, skills with agents and roles, agent names, MCP server rows with their sources, a lexicon, and provenance stating which levels a person supplied and which were generated. An ontology spec holds every structural decision. The builder applies the spec to the packs and either writes every output or writes nothing, enforcing one parent per node, 3–8 concepts per topic, unique ids and labels, a primary agent for every skill, MaxeyN keys that stay permanent across builds, and every link at or above its threshold. The graph carries a SHA-256 digest; saved app state is tied to it.

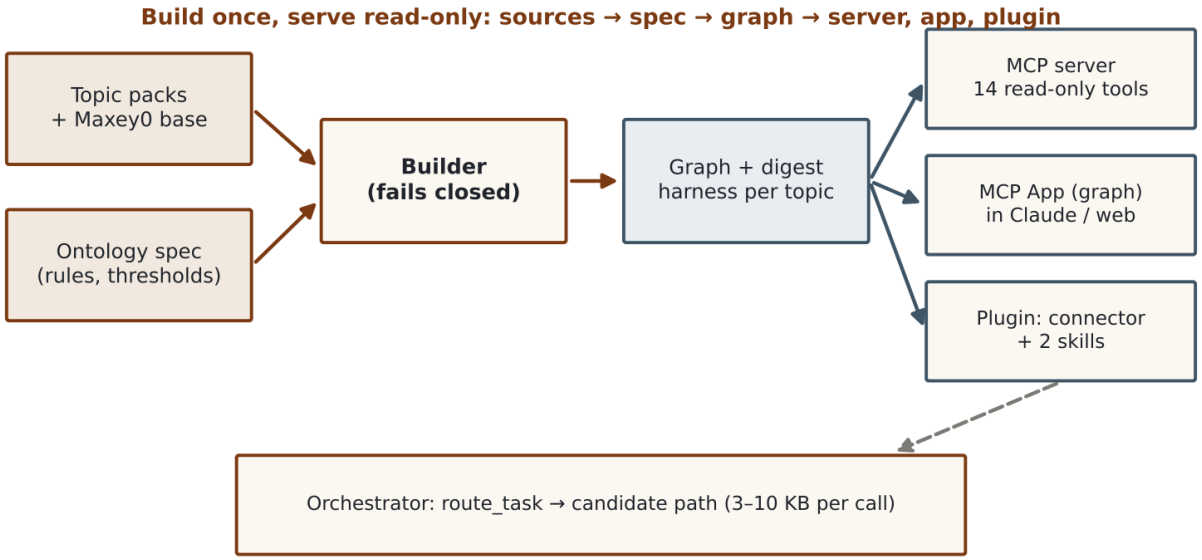


Figure 5. Sources and spec go through a fail-closed builder to one graph with a digest; the server, the app and the plugin read that graph.

Search, routing, agent–server linking and related concepts use one method: TF-IDF cosine with a coverage rule — a pair counts only when it shares at least min(2, terms on the shorter side) words — at thresholds of 0.08 (route, search), 0.10 with the top two agents per server (holds) and 0.40 (related). The server is a TypeScript worker on Cloudflare; its only storage is the dense index (key-value, per build digest) and its only runtime calls embed a task with Workers AI, and with neither available it routes on TF-IDF and says so; it speaks the MCP 2026-07-28 stateless protocol and the earlier initialize handshake, serves the whole plane at `/mcp` and one domain each at eleven scoped endpoints, and annotates all 15 tools read-only.

Tool	Returns
`route_task`	candidate paths: domain → topic → concept → skill → agents by role → servers; flat ranking or the observer lens with its full chain through agents and servers and the drift SCWs; TF-IDF plus the dense signal by default
`nearest`	nearest nodes of any level to a node or text in the shared field, with their 3D field place
`search`	matching nodes at any level, with paths
`list_domains · list_topics · children · trace`	browse the tree; walk any node to its root
`get_topic · get_concept · get_skill · get_agent · get_mcp_server`	one node in detail, links with basis
`get_harness`	one topic complete (about 49 KB)
`open_context_plane_graph`	the graph as an MCP App, routed to a task or opened on a topic
`validate_topic_pack`	problems, warnings and nearest topics for a generated pack

Beside MCP the server answers plain HTTP. `/route` routes a task and `/embedding?key=` returns a node's full bge-m3 vector (1,024 values) with its three nearest nodes on every level, so the values the app shows can be checked outside it. Since 6.7 the same worker is a small site on

Human0's own domain, app.maxey0.com: a front page with the live counts and the ways in, the app at `/app`, reference documentation at `/docs` (each page as HTML, PDF and Markdown) and a research library at `/research` (each paper with a summary, a question-and-answer frame and the PDF). The documentation and papers are static files built from the repository and served beside the worker, so they add no storage or runtime cost.

The response budget is part of the design: about 7 KB of tool definitions, 3 KB for a topic, 5 KB for a skill, 10 KB for a three-path route, 49 KB for a full topic harness. This is just-in-time context [1] applied to the capability map itself.

7 The graph app as an MCP App

The same plane is drawn by a self-contained page that runs on the web and inside Claude as an MCP App. In the host it performs the MCP Apps handshake, takes the task the model passed in, routes it with the server's own index and highlights the path, can request full screen, and can hand a generation request back to the conversation. Levels are planes on one vertical axis in a 3D projection that rotates without limit; nodes sit on rings or at their field locations. A path can be isolated, spread, and dragged, with everything downstream moving on its own level. Agents can be placed at the weighted center of the servers they hold, so agents with similar tools cluster; selecting one lists its candidate gates with score and basis, and toggling a gate moves the agent. Domains are the top plane of the graph, one node per domain linked to its topics. Selecting any node shows its embedding values: its field location, its value as seen from the current observer, and its nearest nodes on every level. The same router as the server runs in the page: a task can be routed flat or through the observer lens, and the readout lists each level's chosen nodes with their weights, signed support and ruled-out candidates. The embedding view follows the observer: from above the graph every node sits at its field location; once a domain, topic, concept or lens route places the observer inside the graph, each plane below it is redrawn from there, with the chosen nodes at the center and every other node as far out as the observer leaves it unexplained. The app and the server compute the same lens chain, checked by a shared test fixture. Every link level can be shown or hidden on its own — domain → topic, topic → concept, concept → skill, skill → agent, agent → server, and related concepts across topics as dashed lines — and the side list browses domains as well as topics, concepts, agents, servers and skills. A selected node can be dragged in 3D: the pointer's movement becomes the world movement parallel to the screen, so it follows the pointer from any angle, stays inside its level's donut and stays where it is dropped. With a lens route and the field view on, drift arrows run from each chosen node's place seen from above to its place seen from the observer, and turn with the graph; nodes are colored on a perceptually even scale by how much of each the observer explains, with a legend. The graph can be operated from the keyboard — along a level, up and down the path, between levels — with a live region reading each node out, and its text meets WCAG AA contrast, checked in the tests. The page routes at once with TF-IDF and then asks the server, whose TF-IDF-plus-dense route replaces the local one; a readout says which signal produced the route. An SCW view draws an agent with its servers as an execution boundary, and several agents together as a loop. A user can generate a missing topic with Claude, validate it, add it and remove it — a generated topic is given the domain of its nearest existing topic, marked as suggested, and the builder files it under the domain named in its pack; upload their own SKILL.md or agent file and see where it maps; and save state tied to the plane's digest, including camera, dragged positions, router mode, observer, domain filter and

every link toggle.

Two features connect the page to Claude without giving it any credential of ours. Connect to Claude offers, by default, to use Maxey0 inside the person's own Claude as a custom connector (the read-only server needs no sign-in), or to use their own Anthropic API key for topic generation on the site. The key is held only in the page's memory: it is not written to browser storage or saved state, is never sent to the Maxey0 server, and goes only to the Anthropic API, whose models the page lists for the person to choose from; reloading the page forgets it. The page's content security policy permits exactly that one outside destination. Send to Claude Code turns the current route into a project: a CLAUDE.md that states the task and the route, an MCP configuration naming the chosen servers, one subagent file per chosen agent in its role, and the route itself as JSON, including the lens drift. The person runs Claude Code in that folder; the plane chooses, the harness executes. The CLAUDE.md leaves a section for retrieval sources open, which is where connected document stores will attach.

Since v1.2 the page also reports what a route does not cover. After each route it splits the task into words, marks those the chosen path explains, those a concept in the map fits but the route ranked lower, and those nothing fits; for the uncovered phrase it walks down the levels with the observer lens and names the level where the map stops fitting: no topic above a TF-IDF score of 0.2 means a new topic in the best domain, a fitting topic without a fitting concept means a new concept under it. Fill the gap turns that into a generation request for only the missing levels; the generated pack is validated, added to the page's routing index (new words take the largest idf), and the task is routed again. This runs without a model up to the request; writing the pack needs one, inside Claude or with the person's own key. The public map changes only when a pack is added to the repository.

> **Why the plane must be visible**
>
> A route is a judgement an operator has to trust. Showing the path, the roles, the basis of every gate and the field location of every node where the route is produced makes a wrong route visible in seconds; the routing gaps reported in Section 9 were found this way.

8 Harnessing the plane: vertical, lateral, tangential

A harness is a fixed slice of the plane prepared for use. The author distinguishes three directions.

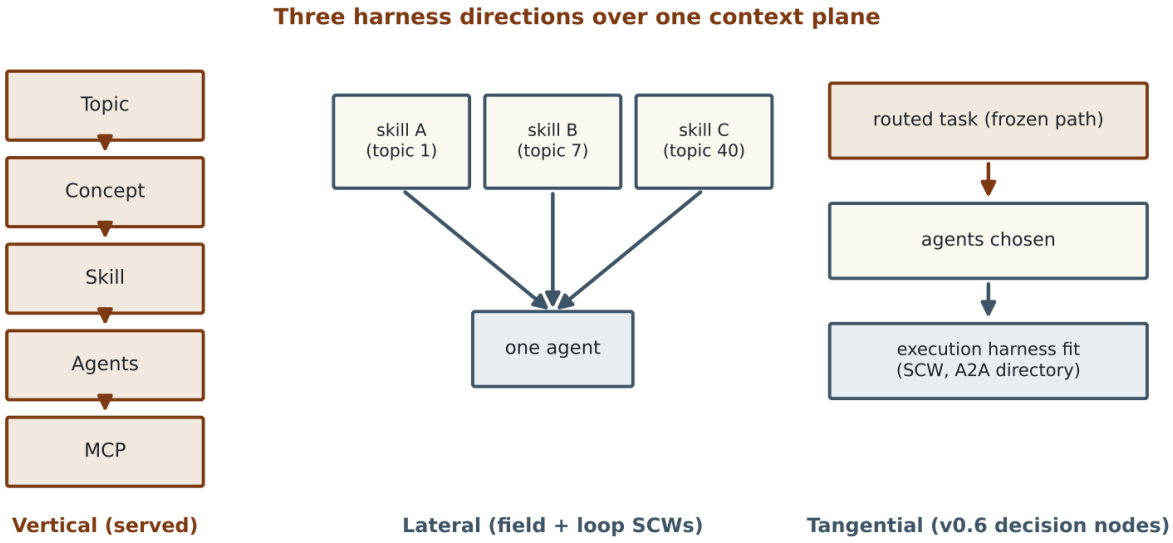


Figure 6. Three harness directions over one context plane.

8.1 Vertical harnesses

A topic harness is one topic from top to bottom: every concept, skill, agent and server under it, with every link. The builder writes one per topic and the server returns one with `get_harness`. Selecting one harness never loads the others, which is what keeps a lookup at kilobytes.

8.2 Lateral harnesses

A lateral harness runs across topics: a multi-skill agent that serves skills in several fields, or an agentic loop of maker and checker agents. Because agents are global and every node lives in the shared field, lateral structure is already present in the data — an agent's skills across topics, the agents nearest a topic from outside it — and the app draws custom loop SCWs that span agents of different skills. Serving a lateral harness as a first-class object, with rules for composing it, is the next step.

8.3 Tangential harnesses: where the plane meets execution

Routing over the plane fixes what the task is and who, in principle, should do it; the resulting path is frozen. A tangential harness is then fitted to it: once particular agents are chosen, the execution boundary — their servers, their model, their runtime, their checker — is shaped to those agents. It is tangential because it touches the plane at one routed path and leaves it toward execution.

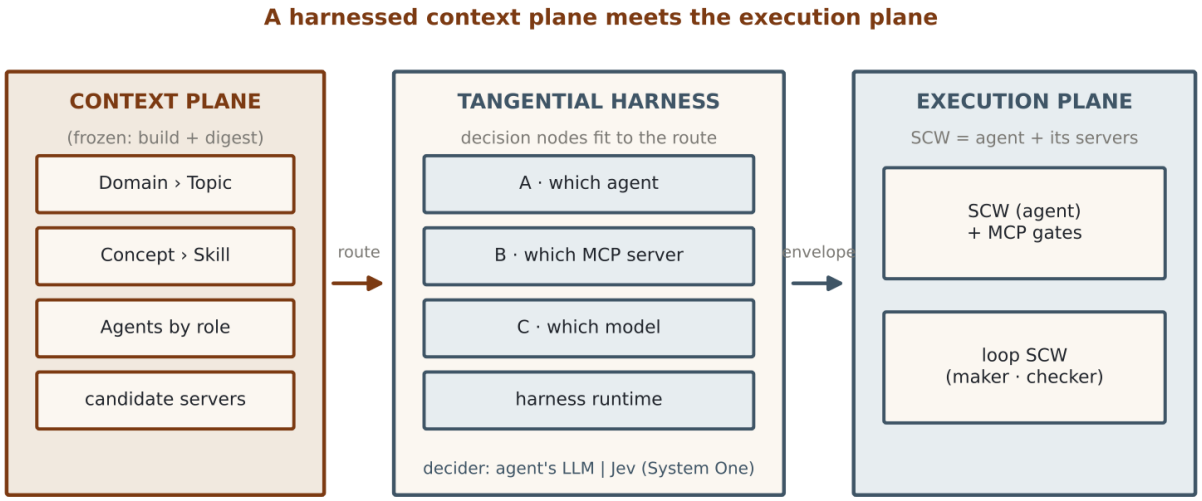


Figure 7. A frozen route from the context plane enters a tangential harness of decision nodes, which emits an envelope for an SCW on the execution plane.

Version 0.6 of the Control Graph implements this bridge as three decision nodes on a routed task: A, which agent runs the task; B, which MCP server it uses, with that agent's gates as candidates; and C, which model powers it. Each node's decider is selectable: the agent's own LLM, or a System One model — Jev 1.13 or Jev 1.14, served by TypeSafe's System One API, which answers typed questions (noul, a probability; choice, a distribution over candidates with a confidence; score, a level on a rubric) over state supplied by the caller [9]. The Context Plane supplies that state. A harness selector names the runtime that will execute the envelope (Claude Agent SDK, OpenAI Agents SDK, LangChain/LangGraph, Google ADK, Claude Code), and the result is exported as a task envelope for an SCW.

Function	Question type	What it does	Authority
A · B · C decision nodes	choice	pick agent, server, model from the plane's candidates	candidate list, never authorization
F1 Re-rank	score	rate each gate on a four-level fit rubric (unrelated · adjacent · partial · direct)	modulates field weights only
F2 Verify	noul	check that a chosen server supports the task	can only downgrade to review or confirm
F3 Triage	noul + choice	detect coverage gaps; propose an SCW topology	a proposal a person applies

One invariant governs all of it: no decider grants admission. A System One answer re-weights, downgrades or proposes; the host decides, and a person can override. Requests are checked against the API contract locally before anything is sent, and with no key configured the nodes report the provider as unavailable and send nothing — both behaviors were exercised in testing. Live, keyed Jev calls and their calibration against LLM decisions are not reported here; the published 1.x app shows the route only, and the decision nodes return in the next version of the

app as the bridge to the Maxey0 orchestrator. An A2A directory of live agents belongs on the same tangent: it says which concrete agents can fill the roles the plane defines.

9 Evaluation

All measurements were run against the deployed server in November 2026. They are small; we state what each shows and does not show.

9.1 Index consistency

We sampled 300 of the 1,998 skills (seed 7) and routed each by its own label against the live server (build 6.5.1).

Router	Skill@1	Skill@5	Topic@1
flat, TF-IDF	0.933	0.997	0.950
flat, TF-IDF + dense (β 0.7)	0.987	1.000	0.993
lens, TF-IDF	0.923	0.977	0.943
lens, TF-IDF + dense (β 0.7)	0.983	0.997	0.990

Three-to-five-word labels recover the exact skill 93.3% of the time with word counts alone and 98.7% with the dense signal: the index is consistent and the skills are distinguishable. Because the queries share the plane's vocabulary, this does not show generalization; the dense signal's gain here is in separating skills whose labels share words.

9.2 Realistic tasks

We wrote 20 everyday tasks across finance, engineering, science, careers and agent orchestration and fixed the acceptable skills for each before any dense run (one task, seed-round valuation, has no skill in the map). With word counts alone flat ranking put a correct skill first on 13 of 19; the lens on 16; with the dense signal flat ranking reaches 15 and has a correct skill in its top three on 17. The table shows the top flat route with word counts alone and what the dense signal changes, in flat ranking and in the lens (build 6.5.1).

Task	TF-IDF top skill	With dense (flat · lens)
price a European call option with Black-Scholes	`dv-option-pricer`	`same · same`
translate a paragraph into French	`euro-source-translation`	`fren-text-translation · same`
trace hallucinations back to their source	`chla-source-translation`	`hallucination-attribution · same`
set up DNS and a CDN for a website	`inf-network-path-tracer`	`DNS checker 2nd · inf-tls-and-dns-checker`
detect anomalies in server logs	`dr-flight-log-analyst`	`anomaly spotter 2nd · ob-log-anomaly-spotter`
explain the Krebs cycle	`hbio-cell-cycle`	`hbio-cell-cycle · hbio-cell-cycle`
analyze bitcoin on-chain flows	`no route`	`cry-token-analyzer · same`
value a startup for a seed round	`no route`	`no route (0.21 < 0.25) · ofr-exemption-selector`

The word-count failures share a cause: server logs shares log with drone flight logs, Krebs cycle shares cycle with the cell cycle, seed-round valuation uses words no skill uses. The dense signal repairs the cases where the right skill is described in other words, and leaves the Krebs cycle, where the map's cell-cycle skill is also semantically close. Bitcoin on-chain flows now reach a token analyzer, which the fixed judgement counts as wrong (the accepted skill was the blockchain explainer) though it is arguably useful. Twenty author-written tasks is a small and possibly biased sample; a held-out, independently written benchmark is required.

9.3 Repair by data

Three dark-matter tasks — searching past conversations for testable ideas about the universe, designing a dark-matter detection experiment, and checking whether a dark-matter hypothesis is falsifiable — routed to a proof sketcher, a political-science survey builder and an ethics review: 0 of 3 useful, because the plane had no concept for the work. Two generated topic packs (Dark Matter and Experimental Cosmology; Conversation Recall and Idea Mining) were added in a separate build, and the same tasks reached the intended skills, 3 of 3, with no code change. The rebuild added no agent–server links: no existing server scored above threshold for the new agents, so tool coverage depends on which servers exist.

10 Formal form: OWL 2 and SHACL

Because the plane is typed, it can be stated in standard ontology languages. We translated it to OWL 2: six pairwise-disjoint classes, functional parent properties under a transitive partOf, role properties under hasAgent, qualified cardinality of exactly one parent, one primary and one checker, a symmetric irreflexive relatedConcept, and reified gate and relation individuals carrying score and basis. Because OWL is open-world, the same constraints are stated as SHACL shapes for closed-world validation.

The translation was rebuilt for build 6.x. It now carries every node's 3D field place and a drift vocabulary — path, level and node SCWs with flat score, ψ, drift, ranks and task norms — whose SHACL rules state that drift equals ψ minus flat, that a level never passes on more task than it received, and that every node a drift record names exists in the plane.

Check (build 6.3.0 graph)	Result
Triples (asserted / after OWL 2 RL closure)	46,704 / 119,115
Live and local graph, nine sections	same digest, 9 of 9 equal
Counts, OWL vs JSON, nine categories	9 of 9 equal
Skill–agent role edges round-trip	6,314 of 6,314
3D field places round-trip	3,986 of 3,986
Checker-inheritance violations (SPARQL)	0
SHACL validation	conforms, 0 violations
Negative control: a skill with two primaries	rejected
A drift record exported from the app	conforms
Negative control: one drift value altered	rejected
OWL 2 RL inconsistency	none

Check (build 6.3.0 graph)	Result
Every skill inferred partOf its domain	1,998 of 1,998

The plane is therefore not a private format: it loads into a triple store, answers SPARQL, validates with standard tools, and the served JSON and the OWL form are the same data. The ontology and shapes are in the repository with the script that builds and checks them; they are not yet published at stable IRIs, and the dense embeddings are not part of the OWL form.

11 Limitations

Limitation	Consequence	Path
Lexical vectors	synonyms and paraphrase miss; shared words in the wrong sense mislead	dense bge-m3 signal added (Section 5.4); the Krebs cycle still fails
Fitted threshold	the flat match threshold 0.25 was chosen on the same 30 requests it is reported on	test on new, independently written requests
Lens has no threshold	an off-map request still receives a lens route	a ψ threshold, calibrated like the flat one
3-D field projection	about 21.5% of the 24-dimension variance; picture neighbors are not always neighbors	rank and drift on full vectors (done)
3-D drift places	depth is compressed relative to the other axes, because a donut is thin	use the exact drift values; the 3D places are a projection
Field vs routing	the 3D field is projected from the TF-IDF space; routing also uses the dense signal	a field projected from the hybrid space
Generated content	concepts, skills, agents generated and structurally checked, not expert-reviewed	per-field review; provenance marks generated levels
Tool coverage	new topics may get no servers	declared links; more real servers
System One deciders	built and contract-tested; live calibration not reported	keyed runs against LLM decisions
Small evaluation	300 self-retrieval queries; 20 author-written tasks	held-out, independently written benchmark

We make no claim that the plane routes better than embedding retrieval alone; we have measured TF-IDF against TF-IDF plus a dense signal, not against a dense-only retriever. Our claim is structural: a typed plane with roles, inheritance, a shared field across levels and a fail-closed build gives properties that retrieval alone does not — explainable routes, a checker on every skill, cross-level proximity, gap detection, repair by data, and a frozen route that an execution harness can be fitted to.

12 Conclusion

The Maxey0 Context Plane separates the map of work from the execution of a task. It places every task on one path through six levels that share one embedding field, serves that plane read-only over MCP with a budget measured in kilobytes, makes it visible and correctable inside the conversation as an MCP App, states it in OWL 2, and hands a frozen route to a tangential harness whose decision nodes can be made by an LLM or by a System One model under a strict no-authority invariant. It grew from a knowledge-format bundle and an isolation primitive, SCW, into a deployed system. The lens now runs from topics to servers, records its own drift as SCWs,

and reads a dense second signal next to word counts. The next steps are a held-out benchmark, run-to-run drift comparison, a field projected from the hybrid space, lateral harnesses as served objects, and the decision nodes returning as the bridge to the Maxey0 orchestrator.

Availability. Site: https://app.maxey0.com (app at /app, documentation at /docs, research at /research). MCP server: https://app.maxey0.com/mcp, also at https://maxey0-context-plane-mcp.maxey0.workers.dev/mcp (read-only, no sign-in). Source, tests and documentation: github.com/mmc7676/Human0, apps/maxey0-context-plane (Apache-2.0). This paper describes Maxey0 Context Plane v1.2.0 - Prefederation (server, graph app, plugin and paper share that version; v1.0.0 was the first public release, internally server build 6.7.1 and app build 1.13). The measurements in Sections 5.4 and 9 were run on server build 6.5.1; builds since then changed the site, the app and the documentation, not the plane, its index or the router, so they still describe the live routes.

Author statement. Mike Cohan (Human0) conceived and directed Maxey0 from the Sportscard App and the Maxey Node Network onward, Structured Context Windows, Maxey0-OKF, the six-level Context Plane Harnessing structure and its shared embedding field, the vertical, lateral and tangential harness model, and the System One decision-node design. Claude (Anthropic) implemented much of the software under his direction, wrote this paper's prose, carried out the literature search and ran the evaluations in Sections 5, 9 and 10. Concepts, skills and agents in the plane were generated with Claude and structurally checked; MCP servers are real.

References

[1] Anthropic. Effective context engineering for AI agents.
anthropic.com/engineering/effective-context-engineering-for-ai-agents.
[2] Anthropic. Equipping agents for the real world with Agent Skills.
anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills.
[3] T. Gan, Q. Sun. RAG-MCP: Mitigating Prompt Bloat in LLM Tool Selection via Retrieval-Augmented Generation. arXiv:2505.03275, 2025.
[4] E. Lumer, A. Gulati, V. K. Subbiah, P. H. Basavaraju, J. A. Burke. ScaleMCP: Dynamic and Auto-Synchronizing Model Context Protocol Tools for LLM Agents. arXiv:2505.06416, 2025.
[5] X. Fei, X. Zheng, H. Feng. MCP-Zero: Active Tool Discovery for Autonomous LLM Agents. arXiv:2506.01056, 2025.
[6] Evolution of AI Agent Registry Solutions: Centralized, Enterprise, and Distributed Approaches. arXiv:2508.03095, 2025.
[7] Model Context Protocol. SEP-1865: MCP Apps — Interactive User Interfaces for MCP. modelcontextprotocol.io/seps/1865-mcp-apps-interactive-user-interfaces-for-mcp.
[8] I. Ong et al. RouteLLM: Learning to Route LLMs with Preference Data. ICLR 2025.
[9] TypeSafe. System One API (Jev). docs.typesafe.ai/api/system-one.
[10] M. Cohan. Structured Context Windows, first public disclosure, 27 September 2025. claude.ai/share/551bdb74-8a71-4b95-a8a7-260241fd760b.